GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering &Technology

Component-Based Software Engineering

Reusable Java Beans: Student Fee Management Module

Student Name	Kothari Tarj Mehulbhai	Enrollment No.	260205020001
Subject	AdvanceSoftwareEngineering (METCE11501)	Semester	1 (M.Tech)
Activity	ALA 1	Marks	10

1. Aim

Toimplementasmallsoftwaremoduleusinga**component-basedarchitecture**with**JavaBeans**,design reusable components with well-defined interfaces, and demonstrate how the components interact inside the system. UML diagrams, code snippets and explanations are included to show the componentstructure.

2. Introduction

Component-Based Software Engineering (CBSE) builds systems by assembling independent, reusable units called **components**. Each component hides its internal implementation and exposes its functionality only through **provided interfaces**, and states what it needs through **required interfaces**. This gives replaceability, reuse and easier maintenance.

A **Java Bean** is a reusable Java component that follows a convention: it has a public no-argument constructor, private properties with public getter/setter methods, implements `Serializable`, and can raise events. *Bound properties* (via `PropertyChangeSupport`) let one bean notify other beans when its state changes, so beans can be wired together without knowing each other's internals.

CBSE concept	How it appears in this module
Component	StudentBean, FeeCalculatorBean, ReceiptBean
Provided interface	FeeCalculator (calculate), add/removePropertyChangeListener, getters/setters
Required interface	ReceiptBeanrequiresthe'fee'propertyevents;FeeCalculatorBeanrequires'course' and 'credits' events
Composition / wiring	Done in Main by registering listeners; no component creates another
Reuse / replacement	Any class implementing FeeCalculator can replace FeeCalculatorBean

3. Tools and Technologies

- Java Development Kit (JDK 17 or later), `java.beans`package
- Any IDE (IntelliJ IDEA / Eclipse / VS Code) or the command line
- Draw.io / PlantUML for UML (the diagrams below are included in this report)

4. Problem Description and Component Design

The module calculates a student's tuition fee and prints a receipt. The work is divided into three looselycoupled components:

Component	Responsibility	Interface exposed
StudentBean	Holdsstudentdata(name,course,credits).Firesevents when course or credits change.	Getters/setters, addPropertyChangeListener()
FeeCalculatorBean	ListenstoStudentBean,computesthefee,andfiresa 'fee' event.	FeeCalculator.calculate(), getFee(), addPropertyChangeListener()
ReceiptBean	Listens to the 'fee' event and generates a receipt.	propertyChange(), setStudent()

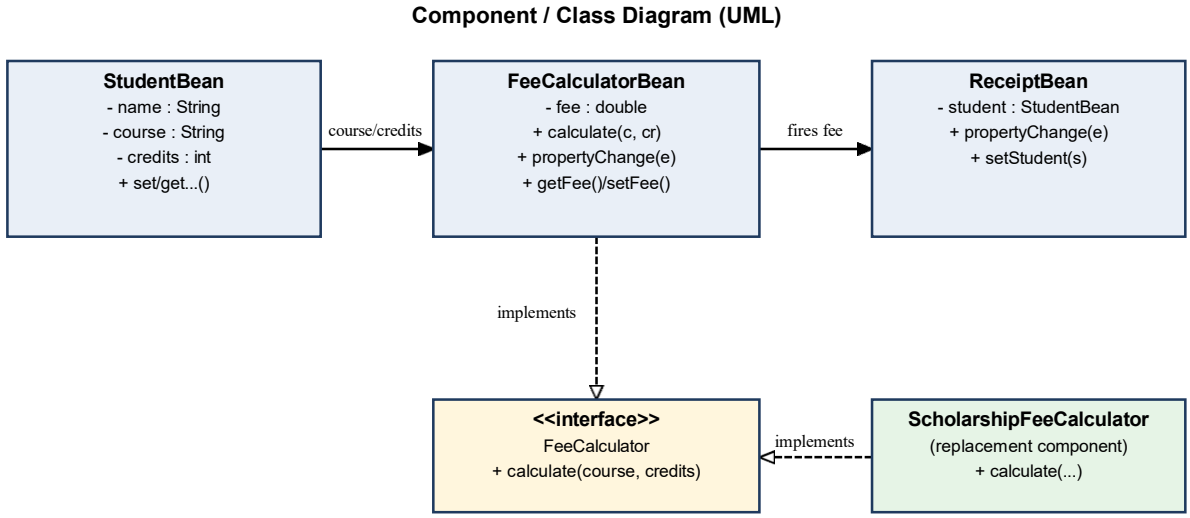


Figure 1: UML class/component diagram showing beans, the provided interface and a replacement component.

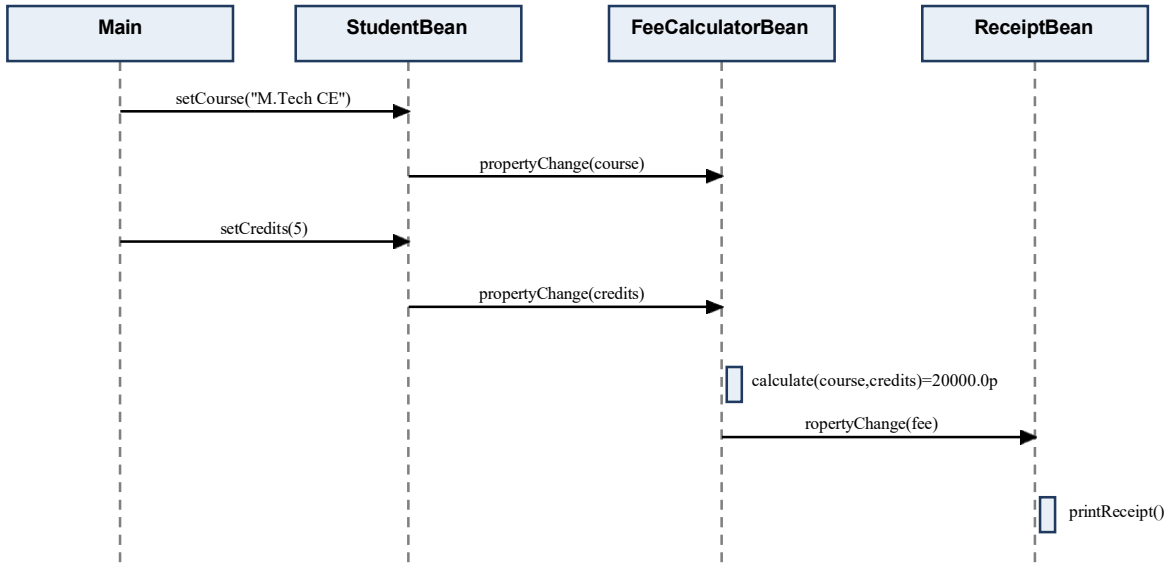


Figure 2: UML sequence diagram of the component interaction when a student's course and credits are set.

5. Implementation

Interface: FeeCalculator.java

```
publicinterfaceFeeCalculator{
    /** Returns the total fee for the given course and number of credits. */
    double calculate(String course, int credits);
}
```

Component 1: StudentBean.java

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.Serializable;

public class StudentBean implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name = "";
    privateStringcourse=""; private
    int credits;
    private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);

    public StudentBean() { }                        // public no-arg constructor

    public String getName() { return name; }
    public void setName(String name) {
        Stringold=this.name;
        this.name = name;
        pcs.firePropertyChange("name",old,name);
    }

    public String getCourse() { return course; }
    public void setCourse(String course) {
        Stringold=this.course;
        this.course = course;
        pcs.firePropertyChange("course",old,course);      //boundproperty
    }

    public int getCredits() { return credits; }
    public void setCredits(int credits) {
        intold=this.credits;
        this.credits=credits;
        pcs.firePropertyChange("credits",old,credits);//boundproperty
    }

    public void addPropertyChangeListener(PropertyChangeListener l)     { pcs.addPropertyChangeListener(l); }
    public void removePropertyChangeListener(PropertyChangeListener l) { pcs.removePropertyChangeListener(l); }
}
```

Component 2: FeeCalculatorBean.java

```
import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.Serializable;

public class FeeCalculatorBean implements FeeCalculator, PropertyChangeListener, Serializable {
    private static final long serialVersionUID = 1L;

    privatedoublefee;
    private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);

    public FeeCalculatorBean() { }

    @Override
    publicdoublecalculate(Stringcourse,intcredits){
        double ratePerCredit = "M.Tech CE".equals(course) ? 4000.0 : 3500.0;
        return ratePerCredit * credits;
    }

    /** Required interface: reacts to changes in StudentBean. */
    @Override
    public void propertyChange(PropertyChangeEvent e) {
        String p = e.getPropertyName();
        if ("course".equals(p) || "credits".equals(p)) {
            StudentBean s = (StudentBean) e.getSource();
            setFee(calculate(s.getCourse(), s.getCredits()));
        }
    }
```

```
    public double getFee() { return fee; }
    public void setFee(double newFee) {
        doubleold=this.fee;
        this.fee = newFee;
        pcs.firePropertyChange("fee",old,newFee);          //notifiesReceiptBean
    }

    public void addPropertyChangeListener(PropertyChangeListener l)     { pcs.addPropertyChangeListener(l); }
    public void removePropertyChangeListener(PropertyChangeListener l) { pcs.removePropertyChangeListener(l); }
}
```

Component 3: ReceiptBean.java

```
import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.io.Serializable;

public class ReceiptBean implements PropertyChangeListener, Serializable {
    private static final long serialVersionUID = 1L;
    privateStudentBeanstudent;

    publicReceiptBean(){}
    publicvoidsetStudent(StudentBeanstudent){this.student=student;}

    @Override
    public void propertyChange(PropertyChangeEvent e) {
        if ("fee".equals(e.getPropertyName())) {
            printReceipt((Double)e.getNewValue());
        }
    }

    private void printReceipt(double fee) {
        System.out.println("=========== FEE RECEIPT ===========");
        System.out.println("Student : " + student.getName());
        System.out.println("Course: " + student.getCourse());
        System.out.println("Credits : " + student.getCredits());
        System.out.println("Fee      : Rs. " + fee);
        System.out.println("==================================");
    }
}
```

Assembling the components: Main.java

```
publicclassMain{
    public static void main(String[] args) {
        StudentBean student = new StudentBean();
        FeeCalculatorBean calculator = new FeeCalculatorBean();
        ReceiptBean receipt = new ReceiptBean();

        // wiring (composition) - components do not know each other's internals
        student.addPropertyChangeListener(calculator);
        calculator.addPropertyChangeListener(receipt);
        receipt.setStudent(student);

        student.setName("Student Name");
        student.setCourse("M.Tech CE");
        student.setCredits(5);           //triggersfeecalculationandreceipt
    }
}
```

Demonstrating reuse: replacing a component

Becausethecalculationisbehindthe`FeeCalculator`interface,adifferentcomponentcanbepluggedin without touching StudentBean or ReceiptBean:

```
public class ScholarshipFeeCalculator implements FeeCalculator {
    @Override
    publicdoublecalculate(Stringcourse,intcredits){
        return0.8*4000.0*credits;                //20%scholarshipdiscount
    }
}
```

Only the wiring in Main changes (the new calculator is used in place of the old one); the other beans stay untouched. This is the main benefit of component-based design.

6. Compilation and Execution

```
javac*.java
java Main
```

Expected output

```
==========FEERECEIPT==========
Student:StudentName
Course: M.Tech CE Credits
: 5
Fee     :Rs.20000.0
==================================
```

Note: the receipt is printed once, when the credits are set. After setCourse() the credits are still 0, so the calculated fee is 0.0 and, because the value did not change, no 'fee' event is fired.

[INSERT SCREENSHOT]

Paste a screenshot of your IDE/terminal showing the compiled project and the output above.

7. Explanation of Component Interaction

- **Encapsulation:**eachbeankeepsitsfieldsprivateandexposesonlygetters/settersandlistener methods.
- **Loosecoupling:**StudentBeandoesnotknowthatacalculatororreceiptexists;itonlyfires property-change events.
- **Event-based communication:** StudentBean →FeeCalculatorBean →ReceiptBean forms a chain of *bound property* notifications (see Figure 2).
- **Interface-based design:** FeeCalculator is the contract; FeeCalculatorBean and ScholarshipFeeCalculator both satisfy it, so they are interchangeable.
- **Reusabilityandserialisation:**allbeansareSerializablewithpublicno-argumentconstructors,sothey can be stored, restored and used by IDE builder tools.

8. Conclusion

A small fee-management module was built from three Java Beans that interact only through defined interfaces and property-change events. The UML class and sequence diagrams document the structure and behaviour. The exercise showed that component-based design improves reusability, replaceability and maintainability compared with a monolithic class in which calculation, data and printing are mixed together.

References

- Ian Sommerville, *Software Engineering*, 9th ed., Pearson Education, 2010 (Chapter on Component-based software engineering).
- Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, 7th ed., 2010.
- Oracle, The Java Tutorials: JavaBeans and `java.beans.PropertyChangeSupport`.